

```
        ❷ document.write("Reached the base case.<br />");
        return;
    } else {
        // RECURSIVE CASE
        ❸ countDownAndUp(number - 1);
        ❹ document.write(number + " returning<br />");
        return;
    }
}
```

```
❺ countDownAndUp(3);
</script>
```

---

When you run this code, the output looks like this:

---

```
3
2
1
0
Reached the base case.
1 returning
2 returning
3 returning
```

---

Remember that every time a function is called, a new frame is created and pushed onto the call stack. This frame is where all the local variables and parameters (such as `number`) are stored. So, there is a separate `number` variable for each frame on the call stack. This is another often confusing point about recursion: even though, from the source code, it looks like there is only one `number` variable, remember that because it is a local variable, there is actually a different `number` variable for each function call.

When `countDownAndUp(3)` is called ❺, a frame is created, and that frame's local `number` variable is set to 3. The function prints the `number` variable to the screen ❶. As long as `number` isn't 0, `countDownAndUp()` is recursively called with `number - 1` ❸. When it calls `countDownAndUp(2)`, a new frame is pushed onto the stack, and that frame's local `number` variable is set to 2. Again, the recursive case is reached and calls `countDownAndUp(1)`, which again reaches the recursive case and calls `countDownAndUp(0)`.

This pattern of making consecutive recursive function calls and then returning from the recursive function calls is what causes the countdown of